

From Pre-LLM Optimization Layer to a
Mathematical Information Operating System

Architecting AMDI-OS and Its Evolution into MIOS: A Unified Account of Structured Context Export for LLM Document Intelligence

AI Development Agent Protocol — Research Scientist & Systems Architect
June 2026 (Merged & Revised Edition)

Abstract

Large Language Models (LLMs) deployed against enterprise document corpora are constrained by fixed token budgets, latency costs, and a structural mismatch between flat-text ingestion and the spatial, tabular, and relational structure native to real documents. This paper presents a unified account of the Adaptive Mathematical Document Intelligence Operating System (AMDI-OS) and its research extension, the Mathematical Information Operating System (MIOS). AMDI-OS establishes a pre-LLM filtration and compression layer: a set of parallel intelligence engines — geometric, statistical, graph-theoretic, and semantic — fused into a single Universal Export Object (UEO) under an explicit token budget, so that the LLM is invoked strictly as a reasoning engine rather than a document parser. MIOS extends this architecture with higher-order mathematics — information-theoretic “Information Physics,” persistent homology, spectral graph analysis, and tensor representations — to capture structural patterns that flat chunking and even graph-based retrieval cannot express. We present the complete architecture, the fusion and budgeting formalism, and a validation framework distinguishing what has been measured from what remains a projected research target. We close with an explicit limitations analysis, since separating verified claims from forward-looking targets is, in our view, a precondition for this line of work being taken seriously rather than dismissed as marketing.

Table of Contents

TableofContents	2
1. Introduction & Engineering Philosophy	4
2. Related Work	5
2.1 Retrieval-Augmented Generation	5
2.2 Graph- and Structure-Aware Retrieval	5
2.3 Mathematical and Topological Document Representations	5
2.4 Positioning	6
3. The Core AMDI-OS Architecture	6
3.1 The Intelligence Engines	6
3.2 Adaptive Fusion and Compression	6
4. The Universal Export Object (UEO) Protocol	7
4.1 Export Priority and Token Budgeting	7
4.2 The Agent Export Layer (AEL)	7
4.3 Post-Generation Verification	8
5. Research Expansion: The Mathematical Information Operating System (MIOS)	8
5.1 Information Physics	8
5.2 Topology and Persistent Homology	9
5.3 Spectral and Wavelet Analysis	9
5.4 Tensor and Hypergraph Frameworks	9
6. System Economics and Multi-Objective Optimization	10
7. Benchmarking, Validation, and Software Architecture	11
7.1 Core Validation Metrics (Methodology)	11
7.2 Evaluation Protocol	11
7.3 Projected Performance Targets	12
7.4 Software Architecture	12
8. Limitations and Threats to Validity	13
8.1 Benchmark Figures Are Not Yet Measured	13
8.2 Fusion Weights Are Heuristic, Not Learned	13
8.3 The “Information Physics” Framing Is an Analogy	13
8.4 Computational Cost of the Mathematical Layer Itself	13
8.5 Relationship to, Not Replacement of, Conventional RAG	13
8.6 Single-Author, Pre-External-Review Status	13
9. Conclusion and Future Work	14
References	15

1. Introduction & Engineering Philosophy

The rapid deployment of generative AI into enterprise workflow has produced a quiet but consequential anti-pattern: using Large Language Models (LLMs) as primary document databases. An LLM is asked to read a contract, a financial report, or a technical manual in full, and to reason directly over the raw text. We argue, consistent with general systems-engineering principles of separation of concerns (in the spirit of SOLID and Clean Architecture), that this conflates two distinct responsibilities—document comprehension and language-based reasoning—inside a single, expensive, and unreliable component.

The scale of the problem is concrete rather than abstract. A standard 200-page PDF report corresponds to roughly 100,000 words, or approximately 130,000 tokens once tokenized for a typical LLM. Submitting that payload directly to a model such as GPT, Gemini, or Claude routinely produces three failure modes:

- **Context pollution**—irrelevant boilerplate (headers, footers, repeated disclaimers) crowds out the signal the model needs.
- **Latency and cost growth**—token count drives both inference latency and per-call cost roughly linearly for most commercial APIs.
- **Hallucination underload**—long, unstructured contexts measurably degrade factual grounding compared to short, structured ones, a pattern consistent with the broader document-intelligence literature on LLM context limitations [1].

Retrieval-Augmented Generation (RAG) is the dominant mitigation: chunk the document, embed the chunks, and retrieve only the top-k segments relevant to a query, typically reducing payload to on the order of a few thousand input tokens. This solves the budget problem but introduces a new one. Naive chunking is agnostic to document structure: a table that spans a page break, a heading hierarchy, a cross-reference between a chart caption and a paragraph two pages later—all of this is flattened or severed by fixed-size or sentence-window chunking. The retrieval step then operates on semantic similarity alone, with no native concept of tabular computation, spatial layout, or graph-structured relationships between entities.

This paper presents AMD-OS as a structural alternative, and MIO-OS as its research extension, organized around one governing principle:

Governing Principle

The LLM should be invoked strictly as a reasoning and language-generation engine, never as a document parser. All structural, statistical, relational, and (in MIO-OS) topological and spectral comprehension of the source document happens in a pre-LLM mathematical layer, which exports a compressed, structured object—not raw text—to the model.

AMD-OS implements this principle through a set of parallel “intelligence engines” (§3) whose outputs are fused (§3.2) into a Universal Export Object under an explicit token budget (§4). MIO-OS (§5) extends the same pipeline with deeper mathematical structure—modeling the document as an information-theoretic physical system, a topological manifold, and a multi-

dimensional tensor — in order to detect structural regularities that the original engine set cannot express. Throughout, we distinguish three categories of claim: (a) architectural decisions that are implemented and testable; (b) mathematical formalisms that are fully specified and internally consistent; and (c) performance figures that remain projected research targets pending empirical validation at scale. §7 and §8 make this distinction explicit, including a dedicated Limitations section, because we think conflating these three categories is the most common and most damaging failure mode in applied AI research writing.

2. Related Work

Document intelligence for LLM pipelines has converged on a small number of dominant strategies, against which AMD-OS and MIOS should be positioned.

2.1 Retrieval-Augmented Generation

Standard RAG architectures retrieve text chunks via dense or sparse embedding similarity and concatenate the top-k results into the model’s context window. This is computationally efficient and has become the de facto industry baseline, but a growing body of work documents its weaknesses on structurally rich documents: tables, multi-column layouts, and cross-page references are systematically degraded by fixed-window or even semantic-boundary chunking. Recent surveys of document intelligence in the LLM era catalog this gap explicitly and call for representations that preserve structural fidelity rather than relying on chunk-level semantic similarity alone [1].

2.2 Graph- and Structure-Aware Retrieval

A second line of work augments retrieval with explicit graph structure — connecting entities, sections, and citations into a knowledge graph that can be traversed rather than only similarity-searched. AMD-OS’s Structural Graph Engine (§3.1) and MIOS’s Hypergraph Framework (§5.4) sit in this lineage, but extend it: PageRank-style centrality identifies importance within a document graph, and hypergraphs allow a single edge to connect more than two structural elements at once (e.g., a header, a table, and its caption simultaneously), which a conventional graph edge cannot represent.

2.3 Mathematical and Topological Document Representations

Persistent homology, spectral graph theory, and tensor decomposition are well established in computational topology and applied linear algebra, but their application to document intelligence — as opposed to image or point-cloud analysis, their more common domains — is comparatively unexplored. MIOS’s contribution (§5) is not the invention of these mathematical tools, but their adaptation to a document-as-manifold framing: treating section hierarchies, layout repetition, and multi-dimensional page/section/row/column relationships as objects these tools can operate on directly.

2.4 Positioning

AMDI-OS and MIOS are best understood as a pre-retrieval structural compression layer that can sit upstream of, rather than in competition with, conventional RAG and graph-RAG pipelines: its output (the UEO, §4) is itself a candidate input to a downstream retrieval or agent system, not a replacement for retrieval. We return to this framing explicitly in the Limitations section (§8), since it materially affects how the benchmark targets in §7 should be interpreted.

3. The Core AMDI-OS Architecture

AMDI-OS intercepts the document ingestion pipeline before any LLM call, using an array of specialized, independently testable intelligence engines to construct a comprehensive digital twin of the source document. MIOS retains this entire layer unmodified and builds its mathematical extensions (§5) on top of it; the two systems share one ingestion pipeline by design, not by coincidence.

3.1 The Intelligence Engines

The system is built as a layered set of services that sequentially normalize and analyze the document along independent dimensions:

- **Geometry & Template Engine** — maps spatial indices, bounding boxes, and alignment, and matches document fingerprints against known structural templates. Rather than exporting exhaustive coordinate data, it isolates only the spatial relationships that affect interpretation (e.g., which caption belongs to which figure).
- **Recurrence & Frequency Engine** — uses rolling hashes, MinHash, and TF-IDF to detect repeating templates (headers, footers, boilerplate clauses), measure text density, and calculate entropy, isolating signal from repeated structural noise.
- **Matrix Engine** — specializes in tabular data: table parsing, aggregation, and statistical correlation, enabling the system to compute derived metrics (e.g., a year-over-year growth rate) natively, before LLM handoff, rather than asking the model to perform arithmetic over a text-rendered table.
- **Structural Graph Engine** — applies PageRank and centrality algorithms to map topological relationships between extracted entities, for example connecting a “Revenue” node directly to the “Expenses” nodes that determine it.
- **Semantic Engine** — generates vector embeddings, performs entity extraction, and produces topic summaries, supplying the similarity-based signal that conventional RAG relies on exclusively, but here as one input among several.

3.2 Adaptive Fusion and Compression

Each engine above operates independently and can fail gracefully without taking down the pipeline — a table-parsing failure does not block semantic extraction, for instance. Once all

available signals are computed, the Adaptive Fusion Engine combines them into a single priority score per extracted object. The fused score is a fixed-weight linear combination:

$$\text{Score} = 0.4S + 0.2M + 0.15G + 0.1X + 0.1F + 0.05R \quad (1)$$

where

S = Semantic relevance, M = Matrix/tabular salience, G = Graph centrality, X = Context/template match, F = Frequency-derived information density, and R = Recurrence.

The weights in Equation 1 were set by engineering judgment to encode a clear prior — semantic relevance dominates, with matrix and graph signals as strong secondary contributors — rather than fitted to a labeled dataset. We flag this explicitly: Equation 1 should currently be read as a principled heuristic prior, not a learned or empirically optimized weighting. Calibrating these weights against human-judged relevance labels is identified as future work in §9.

4. The Universal Export Object (UEO) Protocol

To communicate with downstream AI agents, AMDI-OS and MIOS both use a Universal Export Object (UEO) schema. The UEO guarantees that the LLM receives structured intelligence — explicit Metadata, Summaries, Key Points, Tables, Graphs, Citations, and a system-calculated Confidence Score — rather than an undifferentiated text dump.

4.1 Export Priority and Token Budgeting

Not every extracted object can fit inside a finite context window, so selection is governed by an explicit Export Priority Queue:

$$P = w_1I + w_2C + w_3Q \quad (2)$$

where

I = Object Importance (from Equation 1), C = system Confidence in the extraction, and Q = Relevance to the specific user query, when one is available.

The total payload admitted into the UEO is governed by a Token Budget Manager, which tracks the remaining budget as a simple subtractive ledger:

$$TB = B - U \quad (3)$$

where

TB is the remaining Token Budget, B is the maximum allowed budget for the export, and U is tokens already committed to higher-priority objects. Objects are admitted from the Priority Queue (Equation 2) in descending order until TB reaches zero.

4.2 The Agent Export Layer (AEL)

Different LLMs parse structured context differently, so the UEO passes through an Agent Export Layer that reformats it per target model:

ChatGPT connector — formats the UEO into explicit system prompts, delimited context blocks, and a strict citation array.

Gemini connector — leverages native multimodal structuring, merging text, tables, graphs, and images into a single multimodal payload.

Claude connector — optimizes for long-context capability by emphasizing explicit relationship mappings and exhaustive reference links rather than maximal compression.

This connector layer is the architectural seam where AMDI-OS/MIOS deliberately avoids betting on a single model provider — the mathematical layer is provider-agnostic by construction; only the final formatting step is specialized.

4.3 Post-Generation Verification

A response from the LLM is not trusted by default. AMDI-OS includes a post-generation Response Verification Layer that performs a programmatic fact-verification and citation-verification pass, matching the model's output back against the original UEO contents to check for unsupported claims and broken citation references before the answer reaches the end user. This closes the loop: the same structural data used to compress the input is reused to audit the output.

5. Research Expansion: The Mathematical Information Operating System (MIOS)

MIOS retains the entire AMDI-OS pipeline (§3–§4) and adds a deeper mathematical layer on top of it. The motivating idea is that the engines in §3 capture local, mostly pairwise structure — a table's cells, a graph's edges — but several real document phenomena are not naturally pairwise: a quarterly report's recurring layout across 40 pages, a topic that threads through three disconnected sections, or a table whose meaning depends simultaneously on its page, its section, and its row/column position. MIOS introduces representations designed specifically for this class of higher-order structure.

5.1 Information Physics

MIOS models a document as a physical information system. The baseline Document Information State is a four-tuple:

$$D = (I, E, T, S) \tag{4}$$

where

I is Information content, E is Information Energy, T is Time (document or section recency), and S is Space (position within the document).

Information Energy distinguishes critical content from boilerplate as the product of statistical surprise and topical relevance:

$$IE_i = H_i \times R_i \tag{5}$$

where

H_i is the Shannon entropy of element i , and R_i is its relevance score from the fusion engine (§3.2). High-entropy, high-relevance content (e.g., a specific unusual figure in a financial table) scores far above repeated boilerplate, which is low-entropy by construction.

Important nodes are modeled as attracting nearby information, by analogy with gravitational attraction — an Information Gravity term:

$$G_i = (Importance_i \times Connectivity_i) / Distance_i \quad (6)$$

and the aggregate effect across a document is an Information Field, used as a heatmap for priority retrieval:

$$\Phi = \sum Wq^2 \quad (7)$$

We are explicit that Equations 5–7 are a structured analogy, not a claim that documents obey literal physical conservation laws. Their value is as a principled, single-formalism way to combine entropy, relevance, connectivity, and spatial distance into one retrieval signal — the same role that TF-IDF or PageRank play individually, generalized into one coherent physical metaphor. This framing should be evaluated on retrieval quality, not on physical literalism.

5.2 Topology and Persistent Homology

A document is represented as a topological manifold $\tau(D)$.

$\tau(D)$, constructed from the structural graph of §3.1. By tracking connected components (H_0), loops (H_1), and voids (H_2), MIOS aims to detect persistent structural features — for instance, a topic that splits into two disconnected discussion threads and later reconverges — that survive across multiple scales of analysis and are therefore unlikely to be analysis artifacts. This is the same persistent-homology machinery used in topological data analysis for point clouds, applied here to a document's structural graph rather than to spatial coordinates.

5.3 Spectral and Wavelet Analysis

By computing the eigenvalues and eigenvectors of the document's structural adjacency or Laplacian matrix,

$$AX = \lambda X \quad (8)$$

MIOS detects repeated layout structures — the same eigenvalue signature recurring across a report's monthly sections, for example — without relying on exact-match template detection. A complementary Wavelet Framework $W(a, b)$ captures multi-scale structural variation, where a is a scale parameter and b a translation parameter, allowing the system to detect structural patterns that recur at different granularities (paragraph-level vs. section-level) simultaneously.

5.4 Tensor and Hypergraph Frameworks

Two-dimensional matrices (§3.1) cannot natively represent a value that depends on more than two structural axes at once. MIOS introduces a four-dimensional Tensor Framework:

T_{ijkl}

(9)

indexed simultaneously by Page, Section, Row, and Column, enabling operations such as Tucker or CP decomposition across all four axes jointly — useful, for example, when the same line item recurs across many pages of a report and its trend across pages is itself the object of interest, not just its value on any single page.

Hypergraphs $HG = (V, E)$ extend this further by allowing a single edge to connect an arbitrary number of nodes rather than exactly two — letting MIOS represent, in one structure, the relationship between a table header, a specific data cell, and the caption that explains it, none of which is reducible to a pairwise edge without information loss.

6. System Economics and Multi-Objective Optimization

Both AMDI-OS and MIOS treat computational cost as a first-class, explicitly modeled objective rather than an afterthought. Two efficiency ratios formalize the central trade-off the architecture is designed to optimize:

$$TEC = AnswerQuality / Tokens \quad (10)$$

$$MEC = UsefulData / Memory \quad (11)$$

Token Economics (TEC) and Memory Economics (MEC) express output quality per unit of cost, giving the system a target to optimize toward beyond raw compression: a UEO that is small but produces a poor answer is not, by this metric, an improvement.

The Context Optimization Engine combines these (and related terms) into a single scalar objective to minimize:

$$\text{Minimize } J = w_1 TC + w_2 MC + w_3 ER + w_4 L \quad (12)$$

where

TC = token cost, MC = memory cost, ER = error rate, and L = latency — subject to operating constraints of accuracy > 95%, latency < 2s, and memory < 2GB.

Important Scoping Note

Equation 12's weights (w_1 – w_4) and the constraint thresholds above (95% / 2s / 2GB) are stated as design targets for the optimization objective, not as figures that have been empirically tuned against production traffic. We retain them here because they specify, precisely, what the system is being optimized toward — which is itself useful information — but they should not be read as already-achieved service-level guarantees. See §7 for the full discussion of what is and is not currently validated.

7. Benchmarking, Validation, and Software Architecture

This section is split deliberately into two parts that are easy to conflate but should not be: a methodology that is fully specified and implementable today (§7.1–§7.2), and a table of projected performance targets that the methodology is designed to measure once full-scale evaluation is run (§7.3). Reporting target figures as if they were measured results is, in our view, the single most common integrity failure in applied AI system papers, and we have restructured this section specifically to avoid it.

7.1 Core Validation Metrics (Methodology)

The validation methodology explicitly favors quantifiable metrics over qualitative impression, across four families:

Compression and retention, measuring how much the pipeline shrinks the input while preserving content:

$$TR = AMDI / \text{Baseline}, TI = 1 - TR \quad (13)$$

$$CR = \text{Original} / \text{Compressed}, CP = 1 - (\text{Compressed} / \text{Original}) \quad (14)$$

$$IR = \text{Retained} / \text{Original} \text{ (target threshold: } IR > 0.95) \quad (15)$$

where

TR is Token Reduction Ratio, CR is Compression Ratio, and IR is Information Retention — the architecture’s “north-star” metric, since aggressive compression that discards retrievable information is a regression, not an improvement, regardless of how favorable TR looks in isolation.

Composite performance is tracked via a normalized weighted Benchmark Score:

$$BS = \sum_{i=1}^n w_i M_i \text{ subject to } \sum w_i = 1 \quad (16)$$

Accuracy is assessed with standard retrieval and generation metrics:

$$F1 = 2PR / (P + R) \quad (17)$$

$$\text{Accuracy} = \text{Correct} / \text{Total} \quad (18)$$

$$CA = \text{CorrectCitations} / \text{TotalCitations} \quad (19)$$

7.2 Evaluation Protocol

A methodologically sound evaluation of Equations 13–19 requires, at minimum: (a) a held-out benchmark corpus of structurally diverse real documents (multi-column reports, financial tables, technical manuals) with human-annotated ground-truth answers and citations; (b) a matched baseline pipeline (direct PDF ingestion and a conventional chunked-RAG pipeline) evaluated under identical token and latency budgets; and (c) statistical significance testing across multiple runs, given the variance inherent in LLM sampling. We specify this protocol explicitly because none of the figures in §7.3 below currently reflect such an evaluation having been completed end-to-end — they are the targets the protocol is designed to test against.

7.3 Projected Performance Targets

Status: Projected, Not Measured

Every figure in Table 1 is a design target the architecture is being engineered toward, derived from the structural properties of the compression pipeline (e.g., expected payload size reduction from fusion-based object selection). None of these figures should be read as a result obtained from the evaluation protocol in §7.2 against the baselines specified there. We retain the original projected ranges below because they make the system's design goals concrete and falsifiable — not because they are evidence of achieved performance.

Metric	Direct PDF Baseline	MIOS + AI Agent (Target)
Token Usage	100%	20% – 50% (projected)
Latency	100%	50% – 80% (projected)
Duplicate Data	100%	10% – 30% (projected)
Citation Accuracy	Medium	High, >95% (target)
Table Reasoning	Medium	High (target)

Table 1. Projected performance targets relative to direct-PDF baseline ingestion. All MIOS-column figures are design targets pending the evaluation protocol of §7.2; see §7 and §8 for full discussion.

7.4 Software Architecture

The reference implementation is structured as an enterprise-grade monorepo (amdi-os), separating document intelligence, retrieval systems, and AI connectors into independently deployable services. The backend targets Python 3.12+ with FastAPI, with storage responsibilities deliberately segregated by access pattern rather than handled by one general-purpose database:

- **PostgreSQL** — relational state and transactional metadata.
- **Redis** — caching and short-lived session state.
- **DuckDB** — fast in-process analytics over tabular extracts.
- **Neo4j** — the Structural Graph Engine and, in MIOS, Hypergraph storage.
- **Qdrant** — vector embeddings for the Semantic Engine.

This polyglot persistence approach trades operational simplicity for fitness-for-purpose at each layer — a deliberate engineering trade-off worth naming explicitly, since it raises real deployment and operational-maturity costs that a single-database design would avoid.

8. Limitations and Threats to Validity

We include this section because a paper proposing a compression and reasoning architecture should be held to the same evidentiary standard it asks LLM outputs to meet. The following limitations apply to the present account:

8.1 Benchmark Figures Are Not Yet Measured

As stated in §7.3, Table 1 reports design targets, not measured outcomes. No claim in this paper should be read as evidence that MIOS achieves a 50–80% token reduction or >95% citation accuracy in practice. Establishing that requires the evaluation protocol of §7.2 to be run to completion against a held-out benchmark, which is future work, not completed work.

8.2 Fusion Weights Are Heuristic, Not Learned

The weights in Equation 1 (and, by extension, Equation 2) encode engineering judgment about relative importance, not weights fitted against labeled relevance data. It is plausible that the optimal weighting varies substantially by document domain (financial reports vs. legal contracts vs. technical manuals), and the current fixed-weight formulation does not adapt to that variation.

8.3 The “Information Physics” Framing Is an Analogy

Equations 4–7 (§5.1) are a structured metaphor borrowed from physics, not a claim that documents obey conservation laws or that “information gravity” is a measured physical quantity. Their utility should be judged entirely on downstream retrieval quality, and that judgment has not yet been made empirically.

8.4 Computational Cost of the Mathematical Layer Itself

Persistent homology, eigendecomposition, and tensor decomposition (§5.2–5.4) are themselves computationally nontrivial, with cost that can scale poorly on very large or densely connected documents. The paper does not yet present a complexity analysis or wall-clock benchmark for the MIOS layer in isolation, which is necessary to confirm that the mathematical preprocessing overhead does not erode the latency gains claimed as a target in §7.3.

8.5 Relationship to, Not Replacement of, Conventional RAG

As noted in §2.4, the UEO is naturally positioned as an input to downstream retrieval rather than a wholesale replacement for it. This paper does not present a head-to-head comparison against state-of-the-art graph-RAG systems specifically (as opposed to naive chunked RAG), which is the more demanding and more relevant comparison for establishing a genuine improvement.

8.6 Single-Author, Pre-External-Review Status

This is a systems-architecture paper describing an implemented pipeline and a proposed mathematical extension; it has not undergone independent peer review or third-party reproduction. We flag this not to discount the technical content, but because the appropriate

epistemic confidence in any specific numeric claim here should be calibrated accordingly until external validation occurs.

9. Conclusion and Future Work

AMDI-OS proposes a structural correction to a common anti-pattern in LLM-based document analysis: treating the model as both parser and reasoner. By decomposing comprehension into independent geometric, statistical, graph, and semantic engines, fusing their outputs under an explicit token budget, and exporting a structured object rather than raw text, the architecture aims to let the LLM operate exclusively as a reasoning engine over pre-digested intelligence.

MIOS extends this with Information Physics, persistent homology, spectral analysis, and tensor representations, targeting structural patterns — recurring layouts, multi-axis relationships, topologically persistent themes — that the original engine set, and conventional RAG more broadly, cannot natively express.

The contribution of this paper, beyond the architecture itself, is in being explicit about what is currently established versus aspirational: the engine architecture and UEO protocol are implemented and testable; the MIOS mathematical formalisms are fully specified and internally consistent; the performance targets in Table 1 are projected, not measured. We consider closing that final gap — a completed run of the evaluation protocol in §7.2 against held-out benchmarks and strong baselines — the primary direction for future work, alongside:

- Empirically calibrating the fusion weights (Equation 1) against human-judged relevance labels, rather than relying on engineering priors alone.
- A wall-clock and asymptotic complexity analysis of the MIOS mathematical layer in isolation, to confirm preprocessing overhead does not offset latency gains.
- A direct comparison against graph-RAG baselines specifically, not only naive chunked RAG, as the more demanding relevant comparison.
- External, independent reproduction of both the engine implementations and the benchmark results once §7.2 is executed.

References

- [1] Wang, W., Hu, H., Zhang, Z., Li, Z., Shao, H., & Dahlmeier, D. (2025). *Document Intelligence in the Era of Large Language Models: A Survey*. arXiv preprint arXiv:2510.13366.
- [2] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS) 33.
- [3] Edelsbrunner, H., & Harer, J. (2010). *Computational Topology: An Introduction*. American Mathematical Society.
- [4] Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab Technical Report.
- [5] Kolda, T. G., & Bader, B. W. (2009). *Tensor Decompositions and Applications*. SIAM Review, 51(3), 455–500.
- [6] Edge, D., Trinh, H., Cheng, N., et al. (2024). *From Local to Global: A GraphRAG Approach to Query-Focused Summarization*. Microsoft Research.

Note on references: items [2]–[6] were added in this merged edition to ground the architectural comparisons in §2 (Related Work) and §8 (Limitations) in the established literature on RAG, PageRank, topological data analysis, tensor methods, and graph-RAG. The original AMDI-OS and MIOS drafts cited only [1]; readers should treat [2]–[6] as standard, widely-cited anchor references for the respective subfields rather than as papers AMDI-OS/MIOS has been empirically benchmarked against.